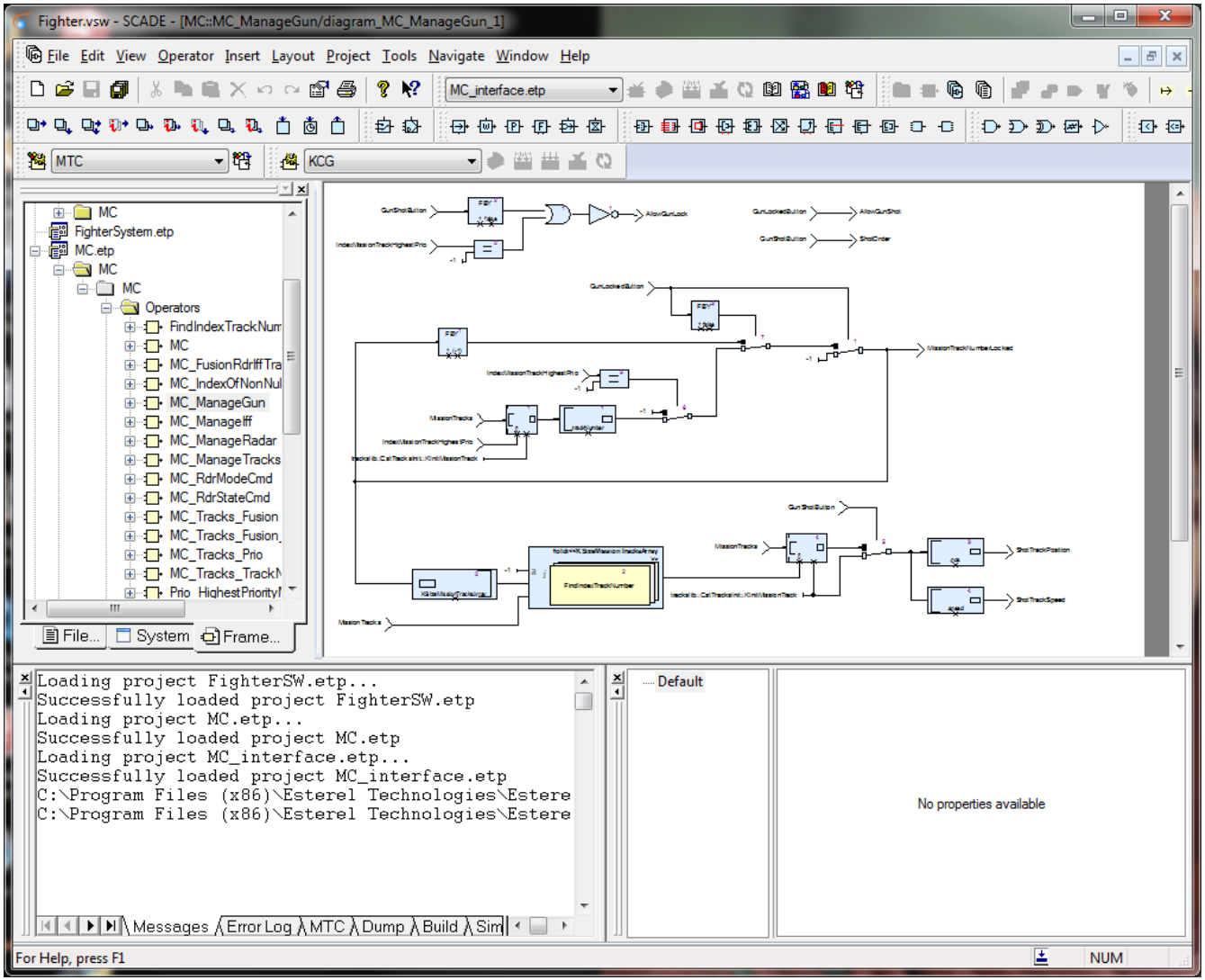
The Scade 6 Language for embedded systems

From design to compilation

Bruno Pagano
Esterel Technologies - ANSYS

Safety Critical Application Development Environment



Some constraints of Embedded Critical

- Reactive system :
  - Intended for real-time OS
  - Bounded memory and WCET
    - no dynamic allocation
    - no recursion

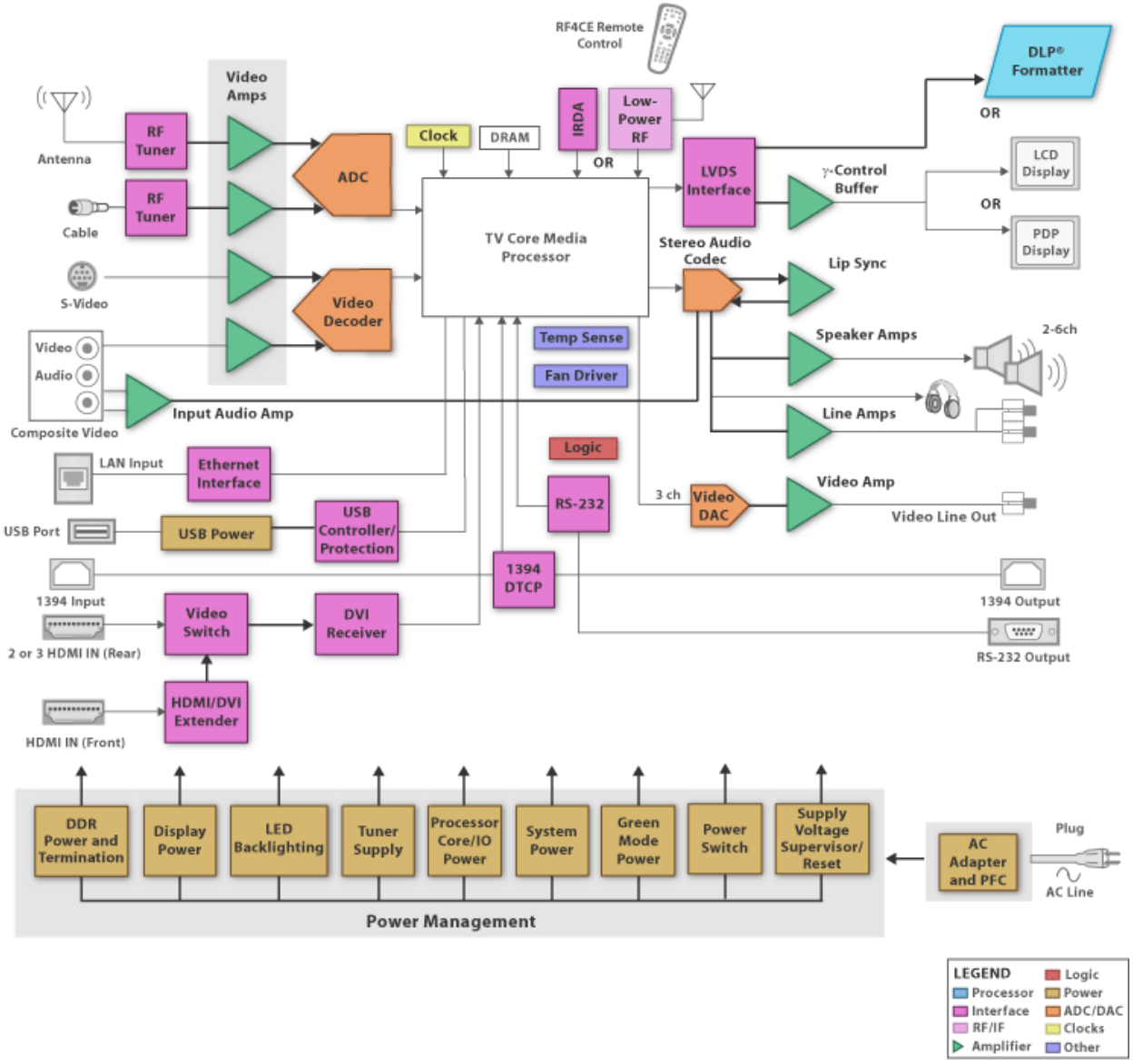
- We expect from the programming language :
  - Exhaustive description
  - Good foundations
  - Determinism
  - Relative simplicity
  - Traceability

Traditional languages

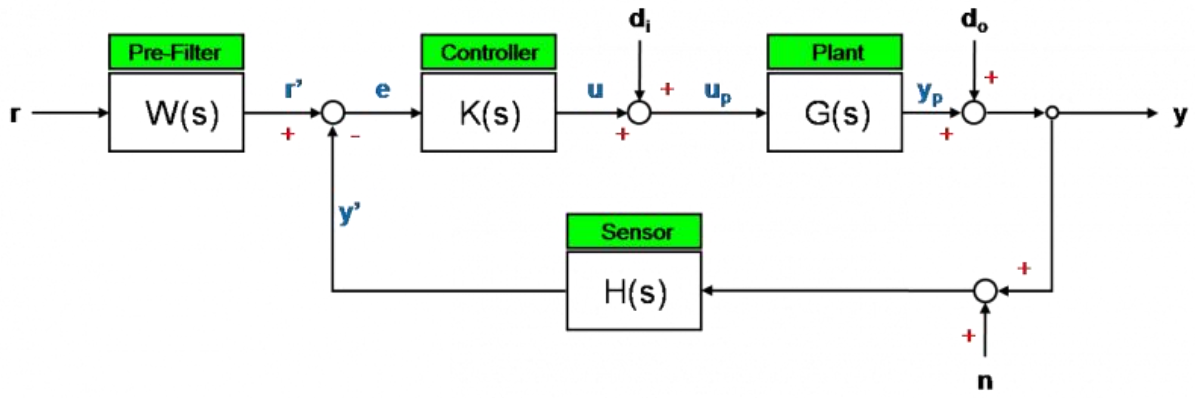
- C and Ada:
  - General and imperative languages
  - Undetermined behaviors :
    - coding rules (MISRA, etc.)
    - checks

- Scade :
  - Dedicated to reactive systems
  - Data flow oriented
  - Very constrained but no undetermined behavior

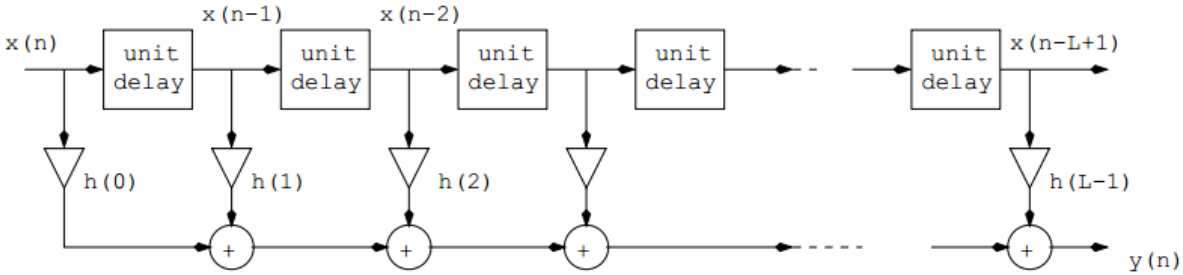
The Block Diagram tradition (Block Diagram)



The Block Diagram tradition (Block Diagram)



The Block Diagram tradition (Block Diagram)



- SAO : Airbus
- Simulink® : Simulation
- SCADE Synchronous Language

  Lustre

  Scade 6

Data flow Language

- At each instant, aftern an initial instant:

Produces a flow (flux) of outputs based on a flow of inputs

$$f : IN^{\mathbb{N}} \rightarrow OUT^{\mathbb{N}}$$
$$f(i_0, i_1, \ldots, i_n, \ldots) = (o_0, o_1, \ldots, o_n, \ldots)$$

- Cycle function (reaction)

$$f : IN \times STATE \rightarrow OUT \times STATE$$
$$f(\overrightarrow{in}_p, state_p) = (\overrightarrow{out}_p, state_{p+1})$$

generated code == cycle function

A very basic example

```
node F (a,b: int) returns (x,y: int)
var
   p : int;
let
  x = a + b ;
  y = if  p<0  then  –p  else  p ;
  p = a – b ;
tel
```

- equations
- principle of substitution
- no order
- here, no state

A very basic example

```
node F (a,b: int) returns (x,y: int)
var
   p : int;
let
  x = a + b ;
  y = if  p<0  then  –p  else  p ;
  p = a – b ;
tel
```

t	t_0	t_1	…	t_n	…			
a	2	2	…	-2	…			
b	1	3	…	-3	…			
x	3	5	…	-5	…			
p	1	-1	…	1	…			
p<0	F	T	…	F	…			
y	1	1	…	1	…			

- equations
- principle of substitution
- no order
- here, no state

A very basic example

```
node F (a,b: int) returns (x,y: int)
var
   p : int;
let
  x = a + b ;
  y = if  p<0  then  –p  else  p ;
  p = a – b ;
tel
```

```
/* ==  input structure  == */
typedef struct { int a; int b; } inC_F;

/* ==  context type  == */
typedef struct { int x;  int y; } outC_F;

/* F */
void F(inC_F *inC, outC_F *outC) {
  int p;

  outC->x = inC->a + inC->b;
  p = inC->a - inC->b;
  if (p < 0) {
    outC->y = - p;
  }
  else {
    outC->y = p;
 }
}
```

- equations
- principle of substitution
- no order
- here, no state

PRE : time delay

y = **pre** x ;

t	t_0	t_1	t_2	t_3	t_4	...		
x	1	2	3	4	5	...		
pre x	nil	1	2	3	4	...		

z = 0 **->** y ;

z	0	1	2	3	4	...		

- Cycle function: STATE

$$f : IN \times STATE \rightarrow OUT \times STATE$$
$$f(X_p, state_p) = (Y_p, Z_p, state_{p+1})$$

PRE : time delay

y = **pre** x ;

z = 0 **->** y ;

t	t_0	t_1	t_2	t_3	t_4	...		
x	1	2	3	4	5	...		
pre x	nil	1	2	3	4	...		
z	0	1	2	3	4	...		

- Cycle function : STATE = Bool x Int

$$state_O = (init_0, memX_0) = (true, nil)$$

$$Y_p = memX_p$$

$$Z_p = \begin{cases} 0 & si\ init_p \\ Y_p & sinon \end{cases}$$

$$init_{p+1} = false$$

$$memX_{p+1} = X_p$$

PRE : time delay

```
/* ==  no input structure  == */


/* =====  context type  ========= */
typedef struct {
   /* -------  outputs  ---------------------- */
   int Y;
   int Z;
   /* ------- initialization variables  ------ */
  bool init;
  /* ------- local memories  -------------- */
   int rem_X;

} outC_Pre;
```

```
void Pre_reset(outC_Pre *outC)
{
 outC->init = kcg_true;
}


void Pre(int X, outC_Pre *outC)
{
  outC->Y = outC->rem_X;
  if (outC->init) {
    outC->init = kcg_false;
    outC->Z = 0;
  }
  else {
    outC->Z = outC->Y;
  }
  outC->rem_X = X;
}
```

Example : natural numbers

```
node POS () returns (x: int)
  x = 1 -> 1 + pre x ;


node NAT () returns (x: int)
  x = 0 -> POS() ;
```

- A Scade node may not have any entries
- Principle of substitution
- Expansion / Non-expansion

WHEN: subsampling of a flow

X = NAT() ;

C = (X **mod** 3 = 0) ;

Y = (2 * X) **when** C ;

t	t_0	t_1	t_2	t_3	t_4	t_5	t_6	…
X	0	1	2	3	4	5	6	…
2 * X	0	2	4	6	8	10	12	…
C	T	F	F	T	F	F	T	…
Y	0			6			12	…

- **Y** is only defined at certain instants : at clock **C**
- Undefined flow is different from nil (value not determined)

WHEN: sub-sampling

```
X = NAT() ;
C = (X mod 3 = 0) ;
Y = (2 * X) when C ;
```

t	t_0	t_1	t_2	t_3	t_4	t_5	t_6	…
X	0	1	2	3	4	5	6	…
2 * X	0	2	4	6	8	10	12	…
C	T	F	F	T	F	F	T	…
Y	0			6			12	…

- **Y** is only defined at certain instants : at clock **C**
- Undefined flow is different from nil (value not determined)

~~Z = X + Y ;~~

- This equation is illegal

WHEN: sub-sampling

```
X = NAT() ;
C = (X mod 3 = 0) ;
Y = (2 * X) when C ;
```

t	t_0	t_1	t_2	t_3	t_4	t_5	t_6	…
X	0	1	2	3	4	5	6	…
2 * X	0	2	4	6	8	10	12	…
C	T	F	F	T	F	F	T	…
Y	0			6			12	…

- **Y** is only defined at certain instants : at clock **C**
- Undefined flow is different from nil (value not determined)

~~Z = X + Y ;~~

- This equation is illegal

```
Z = current Y ;
```

Y	0			6			12	…
Z	0	0	0	6	6	6	12	…

Causality

- Problem:

  Establish that the program determines one and only one value for all the flows.

- Simple and radical solution :
  - The equations are directional
  - Every flow must have one and only one definition
  - No loop that does not pass through a PRE

- Justification :
  - The requirement of the SCADE application domain
  - The requirement of SCADE users

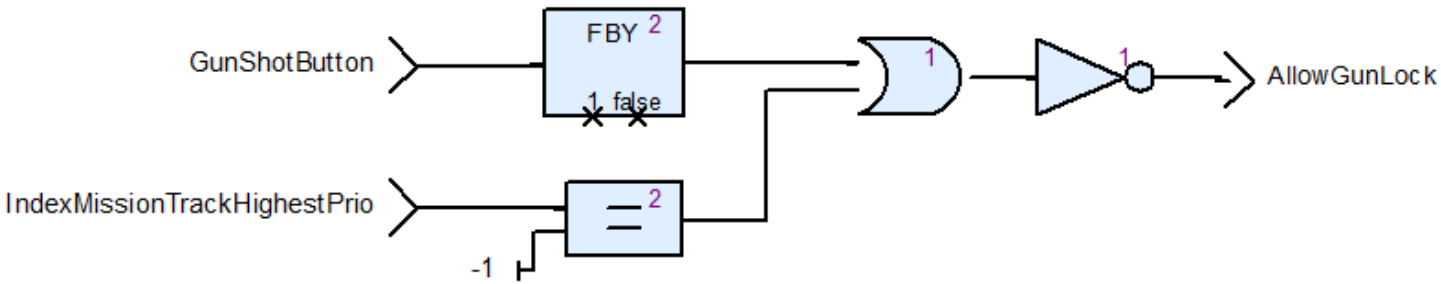
- In practice, it is a triviality :
  - The equations define the relation « is defined by»
  - We verify that this relation is a strict order

Scade Compilation (KCG)

- Converting graphic to text
- Reading and checking the program:
  - Syntax
  - Certain “general” properties
  - Types
  - Clocks
  - Causality (Scade 6)
  - Initialization of flows (Scade 6)
- Diving into the kernel language
- Expansion (inlining)
- Simplifications (optimizations)
- Ordering equations and declaring memories
- Transformation in imperative language
- ….
- C/Ada code generation

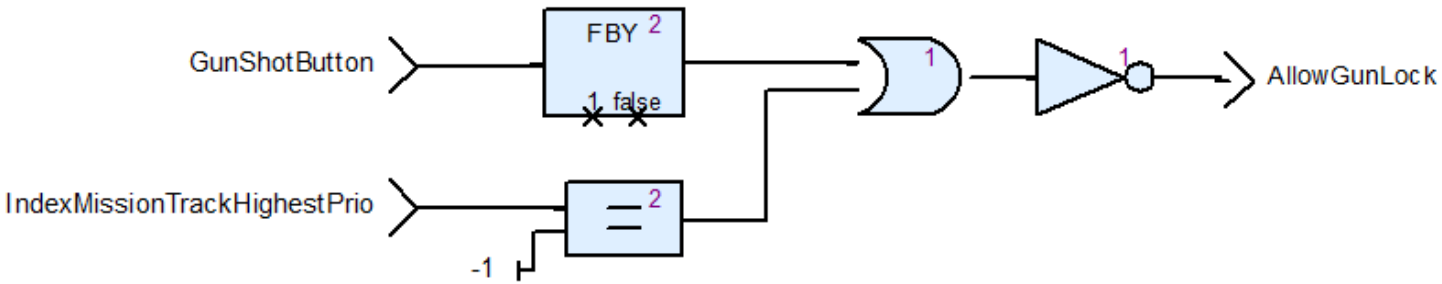
Graphic to Text

Any “Box” is interpreted as an equation.



Graphic to Text

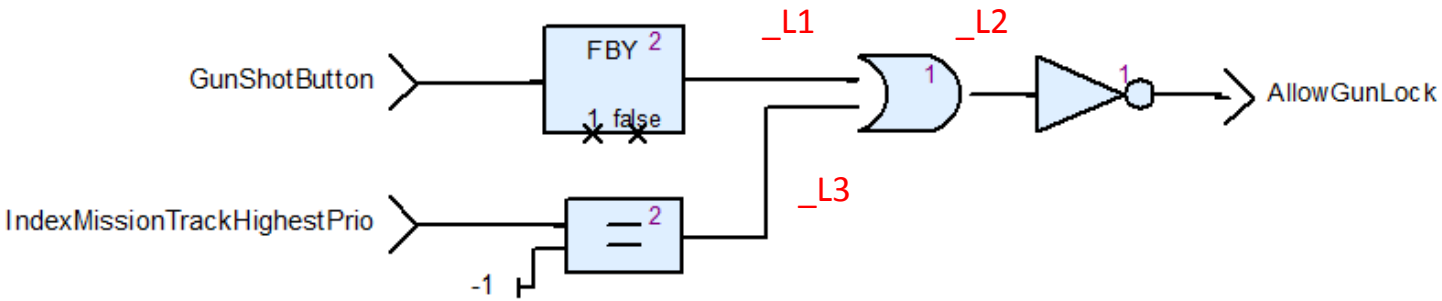
Any "Box" is interpreted as an equation.



AllowGunLock = **not** (**fby** (1; false; GunShotButton) **or** (IndexMissionTrackHighestPrio = -1)) ;

Graphic to Text

Any "Box" is interpreted as an equation.



AllowGunLock = **not** (**fby** (1; false; GunShotButton) **or** (IndexMissionTrackHighestPrio = -1)) ;

_L1 = **fby** (1; false; GunShotButton) ;

_L2 = _L1 **or** _L3 ;

AllowGunLock = **not** _L2 ;

_L3 = IndexMissionTrackHighestPrio = -1 ;

Scade Compilation (KCG)

- Converting graphic to text
- Reading and checking the program:
  - Syntax
  - Certain “general” properties
  - Types
  - Clocks
  - Causality (Scade 6)
  - Initialization of flows (Scade 6)
- Diving into the kernel language
- Expansion (inlining)
- Simplifications (optimizations)
- Ordering equations and declaring memories
- Transformation in imperative language
- ....
- C/Ada code generation

Expansion

- Purpose: replace a node with its equations
- Problem: maintain traceability of generated code
- Scade: expansion is optional

- Why expand or not expand?
  - Performance: works both ways !
  - Solve causality problems

Expansion and causality

- Solve causality problems

```
node F(a,b: int) returns (x,y: int)
  x , y = ( a + 1 , 0 -> pre b) ;

node Test (a:int) returns (x:int)
var
  aux : int ;
let
  aux , x = F (a, aux) ;
tel
```

```
node Test (a:int) returns (x:int)
var
  aux : int ;
  F_a, F_b, F_x, F_y : int;
let
  F_a = a ;
  F_b = aux ;
  aux = F_x ;
  x = F_y ;

  F_x, F_y = (F_a + 1, 0 -> pre F_b) ;
tel
```

After simplification :

```
aux = a + 1 ;
x = 0 -> pre aux ;
```

Scade Compilation (KCG)

- Converting graphic to text
- Reading and checking the program:
  - Syntax
  - Certain “general” properties
  - Types
  - Clocks
  - Causality (Scade 6)
  - Initialization of flows (Scade 6)
- Diving into the kernel language
- Expansion (inlining)
- Simplifications (optimizations)
- Ordering equations and declaring memories
- Transformation in imperative language
- ....
- C/Ada code generation

Scheduling

- Causality :

  There is an order on the equations for which every flow is written “before” being read.

  There are multiple and they are not equivalent for the code that will be generated.

```
X = NAT() ;
Y = 1 + pre X ;
Z = X + 1 ;
```

X is causally related to Z.
Y is independent of X and Z.
3 sequences are posible.

Scheduling

- Causality :

  There is an order on the equations for which every flow is written “before” being read.

  There are multiple and they are not equivalent for the code that will be generated.

```
X = NAT() ;
Y = 1 + pre X ;
Z = X + 1 ;
```

X is causally related to Z.
Y is independent of X and Z.
3 sequences are posible.

```
Y = 1 + pre X ;
X = NAT() ;
Z = X + 1 ;
```

```
Y = 1 + mem_X ;
X = NAT() ;
Z = X + 1 ;
…
mem_X = X ;
```

```
Y = 1 + mem_X ;
mem_X = NAT() ;
Z = X + 1 ;
```

Scade Compilation (KCG)

- Converting graphic to text
- Reading and checking the program:
  - Syntax
  - Certain “general” properties
  - Types
  - Clocks
  - Causality (Scade 6)
  - Initialization of flows (Scade 6)
- Diving into the kernel language
- Expansion (inlining)
- Simplifications (optimizations)
- Ordering equations and declaring memories
- Transformation in imperative language
- …
- C/Ada code generation

Power of expression

- Can we calculate ABRO ?
  - Yes … but …

- Why do you want to expand Scade ?
  - Further solidify the foundations of the language
  - Scaling up (software engineering, tables)
  - Introduce more « control»

Expand Scade under what conditions ?

- Do not lose the fundamental properties :
  - Conservative extension
  - Data flow orientation
  - Traceability of the generated code
  - Correction analyzes present in Scade

Spelling reform

- Harmonize/simplify Scade syntax

```
node Plus (a,b: int) returns (x: int)  ;
let
   x = a + b ;
tel
```

- Anecdotal but :
  - Non-ambiguity
  - Consistency: simplicity, readability, usability
  - Acceptance of the language by users
  - Embedded criticism involves a lot of reviews

Checking the initialization

```
X = NAT() ;
Y = pre X ;
Z = Y -> 1 + pre Z ;
```

t	t_0	t_1	t_2	t_3	...
X	0	1	2	3	...
Y	nil	1	2	3	...
Z	nil	nil	nil	nil	...

- We accept that certain flows are locally undetermined at certain instants in programs which are globally deterministic.

- An undefined flow must reach neither an output nor a memory.

Checking initialization by typing

- 2 types :
  - **0** : type of flows initialized without delay
  - **1** : type of flows initialized with a delay of 1 tick
  - with $\mathbf{1} \sqcup \delta = \mathbf{1}$ et $\mathbf{0} \sqcup \delta = \delta$

$$\frac{e \ : \ \mathbf{0}}{\mathbf{pre}\ e \ : \ \mathbf{1}} \qquad \frac{e_1 : \delta_1 \qquad e_2 : \delta_2}{e_1 \rightarrow e_2 \ : \ \delta_1} \qquad \frac{e_1 : \delta_1 \qquad e_2 : \delta_2}{e_1 + e_2 \ : \ \delta_1 \sqcup \delta_2}$$

- voir J-L. Colaço, M. Pouzet : Type-based Initialization Analysis of a Synchronous Data-flow Language.

Checking initialization by typing

- 2 types :
  - **0** : type of flows initialized without delay
  - **1** : type of flows initialized with a delay of 1 tick
  - with $\mathbf{1} \sqcup \delta = \mathbf{1}$ et $\mathbf{0} \sqcup \delta = \delta$

$$\frac{e \ : \ \mathbf{0}}{\mathbf{pre}\ e \ : \ \mathbf{1}} \qquad \frac{e_1 : \delta_1 \qquad e_2 : \delta_2}{e_1 \rightarrow e_2 \ : \ \delta_1} \qquad \frac{e_1 : \delta_1 \qquad e_2 : \delta_2}{e_1 + e_2 \ : \ \delta_1 \sqcup \delta_2}$$

```
X = NAT() ;
Y = pre X ;
Z = Y -> 1 + pre Z ;
```

$$\frac{\dfrac{X = \mathrm{NAT}() : \mathbf{0}}{\mathrm{Y} = \mathbf{pre}\ \mathrm{X} : \mathbf{1}} \qquad \dfrac{1 : \mathbf{0} \qquad \dfrac{\mathrm{Z} : \mathbf{0} = \delta}{\mathrm{pre\ Z} : \mathbf{1}}}{1 + \mathrm{pre\ Z} : \mathbf{0} \ \sqcup \ \mathbf{1} = \mathbf{1}}}{\mathrm{Z} = \mathrm{Y} \ \rightarrow 1 + \mathrm{pre\ Z} : \mathbf{1} = \delta}$$

- voir J-L. Colaço, M. Pouzet : Type-based Initialization Analysis of a Synchronous Data-flow Language.

Checking Causality

- Check that there is no cycle connecting a variable to itself.

- This is not satisfactory for expanded nodes.

- Verification of causality by typing.

  - Voir P. Cuoq and M. Pouzet : Modular Causality in a Synchronous Stream Language. (*ESOP'01)*

Checking causality by typing

```
node F(a,b: int) returns (x,y: int)
  x , y = ( a + 1 , 0 -> pre b) ;

m , n = F (p, m) ;
```

F non-expanded : $\forall \gamma_1 \gamma_2 . \gamma_1 \times \gamma_2 \rightarrow \gamma_1 \cup \gamma_2$

F expanded : $\forall \gamma_1 \gamma_2 \gamma_3 . \gamma_1 \times \gamma_2 \rightarrow \gamma_1 \times \gamma_3$

m,n : $\gamma_m \times \gamma_n$ et p,m : $\gamma_p \times \gamma_m$

F non-expanded : m,n = F(p,m) : $\gamma_m \times \gamma_n < \gamma_p \cup \gamma_m$ illegal

F expanded : m,n = F(p,m) : $\gamma_m \times \gamma_n < \forall \gamma_3 . \gamma_p \times \gamma_3$ legall

Introduction of « Packages »

- Goal :
  - Organize the source program by grouping certain parts.
  - Facilitate the creation of libraries.
  - Restrict the visibility of certain parts.
  - Provide point scoring and visibility guidelines.

- The view given in SCADE is not ordered.

- We do not set masking rules; we prohibit any ambiguous masking (at the same level).

Polymorphism

- Purpose: similar to « package »
  - « Write Things Once »
  - Reusability

```
node OnlyOn (a: ‘T; c: bool) returns (x: ‘T)
let
  x = if c then a else pre x ;
tel
```

- The constraints do not come from Scade but from the target language:
  - No polymorphism in C and the **void*** type is strictly for bidden (as are most casts).

Restriction on polymorphism

- A Scade program can contain polymorphic parts but must be monomorphic !

- In practice, a Scade program must not contain free type variables after typing.

Arrays in Scade 5

- Arrays exist but not loops !
- No dynamic indexing either.

```
node Sum (a: int^5) returns (x: int)
let
  x = a[0] + a[1] + a[2] + a[3] + a[4] ;
tel
```

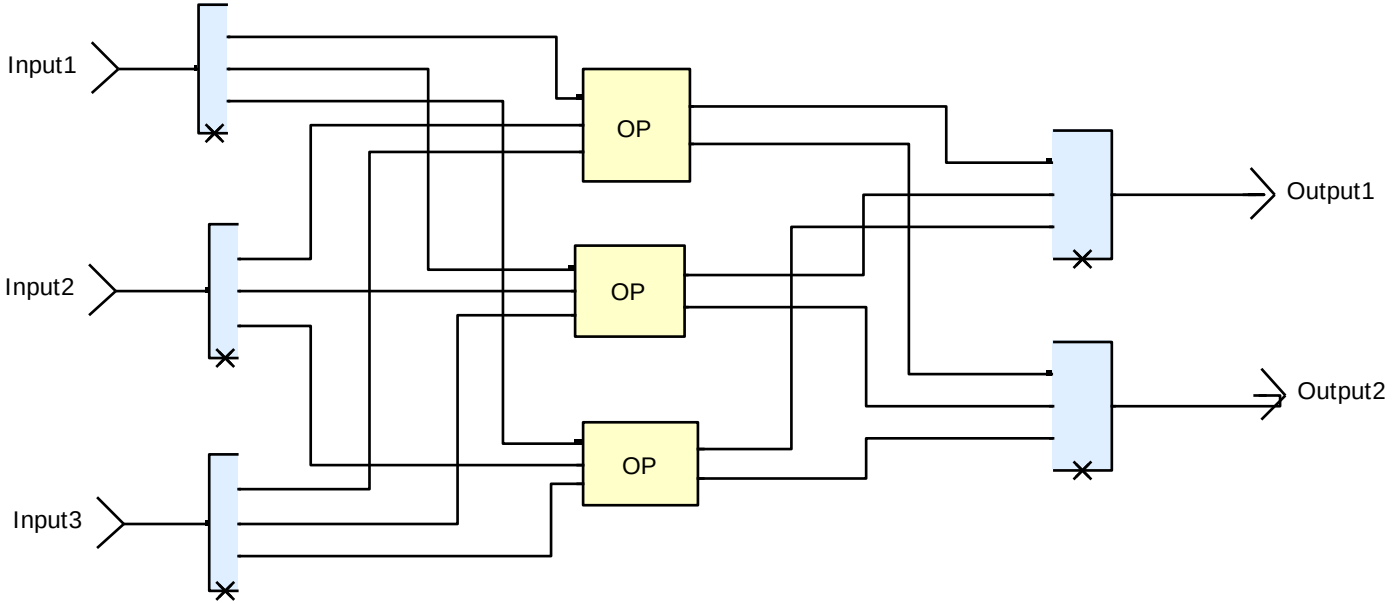
```
node VectOfInt (a: int) returns (x: int^5)
let
  x = [ a, a, a, a, a] ;
tel
```

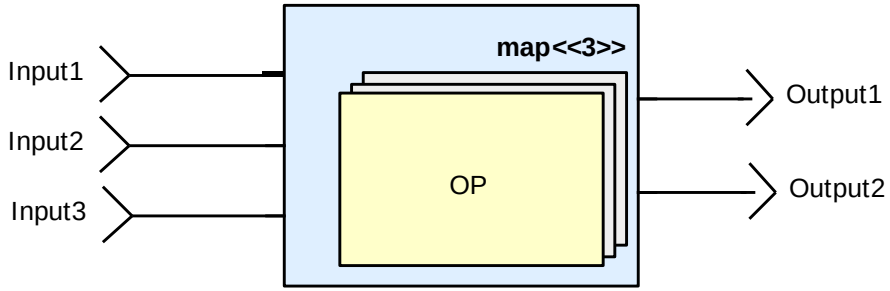
- Manipulating an array of size N requires a pro gram of size N.

Draw ‘for’ loops ?

- How can we give them a « data flow» meaning?
- How to write a loop graphically ?
- How to stay within the « on» frame?
- How to generate efficient and readable code ?
  - presentation at Synchron ‘99 by N. Halbwachs and J-L. Colaço;
  - thesis of L. Morel
- higher order combinators (iterators)
- some additional operators
- Dynamic indexing with a default

MAP : data flow vision





MAP

$$\text{OP} : I_1 \times \cdots \times I_p \rightarrow O_1 \times \cdots \times O_q$$

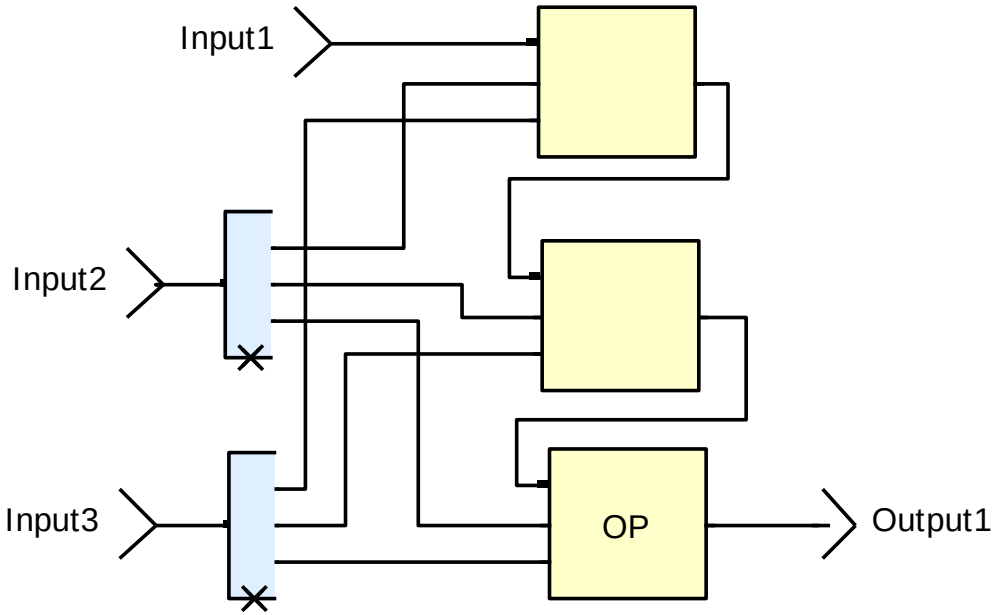
$$(\textbf{map}\ \text{Op} \langle\langle n \rangle\rangle) : {I_1}^n \times \cdots \times {I_p}^n \rightarrow {O_1}^n \times \cdots \times {O_q}^n$$

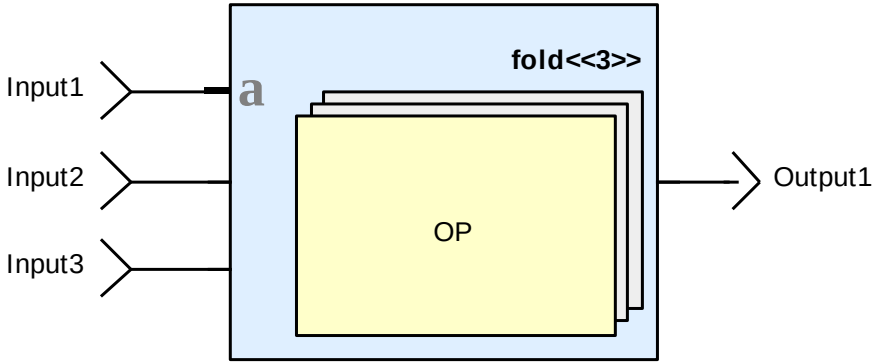
```
Y1, …, Yq = (map Op <<n>>) (X1, …, Xp) ;
```

```
Y1[0], …, Yq[0] =  Op(X1[0], …, Xp[0])
Y1[1], …, Yq[1] =  Op(X1[1], …, Xp[1])
……..……..……..……..……..……..……..……..……..……..
Y1[n-1], …, Yq[n-1] =  Op(X1[n-1], …, Xp[n-1])
```

```
for (i=0; i<n; i++) {
   Op (X1[i], …, Xp[i], &Y1[i], …, &Yq[i]) ;
}
```

FOLD : data flow view





FOLD

$$\text{OP} : \text{ACC} \times I_1 \times \cdots \times I_p \rightarrow ACC$$

$$(\textbf{fold}\ \text{Op} \langle\langle n \rangle\rangle) : \text{ACC} \times {I_1}^n \times \cdots \times {I_p}^n \rightarrow ACC$$

```
Z = (fold Op <<n>>) (X, Y1, …, Yp) ;
```

```
Z = Op( … (Op (Op(X, Y1[0], …, Yp[0]),  Y1[1], …, Yp[1])) … ), Y1[n-1], …, Yp[n-1]) ;
```

```
 Z= X ;
 for (i=0; i<n; i++) {
    Z = Op (Z, Y1[i], … , Yp[i]) ;
}
```

Arrays: Other constructions

- Mapfold, mapfoldi, mapfoldw, mapfoldwi, …

- Create a array : `Z = X ^ n ;` `Z = [ X, X, …, X ] ;`

- Extract from a array : `Z = X [ n .. p ] ;` `Z = [ X[n], X[n+1], …, X[p] ] ;`

- Concatenating arrays : `Z = X @ Y ;` `Z = [ X[0], ..,X[n],Y[0] , …, Y[p] ] ;`

- Transpose of arrays : `Z = transpose ( X ; n ; p ) ;`

  creates a copy array of X but where the dimensions n and p are exchanged

- Dynamic indexing : `Z = ( X . [ a ] default b) ;`

- Copy with modification : `Z = ( X with [ a ] = b) ;`

Compilation of arrays

- Postulate :
  - obtain ‘for’ loops
  - do not break the data flow nature.
- An iterator is compiled by a loop
- Similar to processing structures.
- Causality :
  - Restriction linked to the generated form

The product of two matrices: Dot product

```
function prod_sum (a,v,w:  real) returns (aa: real)
  aa = a + v*w;

-- scalar product of two vectors: V . W
function ScalProd (V,W: real^n) returns (sp: real)
  sp = (fold prod_sum <<n>>) (0.0, V, W);
```

```
 /* prod_sum */
kcg_real prod_sum(
   /* prod_sum::a */ kcg_real a,
   /* prod_sum::v */ kcg_real v,
   /* prod_sum::w */ kcg_real w)
{
   /* prod_sum::aa */ kcg_real aa;

   aa = a + v * w;
   return aa;
}
```

```
 /* ScalProd */
kcg_real ScalProd(
   /* ScalProd::V */ array_real_3 *V,
   /* ScalProd::W */ array_real_3 *W)
{
   kcg_int i;
   /* ScalProd::sp */ kcg_real sp;

   sp = 0.0;
   for (i = 0; i < 3; i++) {
      sp = /* prod_sum */ prod_sum(sp, (*V)[i], (*W)[i]);
   }
   return sp;
}
```

The product of two matrices: matrix x vector

```
-- product of a matrix by a vector: A * u
function MatVectProd (A: real^m^n; u: real^n)  returns  (w: real^m)
   w = (map ScalProd <<m>>)(transpose (A; 1; 2), u^m);
```

```
 /* MatVectProd */
void MatVectProd(
   /* MatVectProd::A */ array_real_4_3 *A,
   /* MatVectProd::u */ array_real_3 *u,
   /* MatVectProd::w */ array_real_4 *w)
{
  array_real_3_4 tmp;
  kcg_int i1;
  kcg_int i;

  for (i = 0; i < 3; i++) {
     for (i1 = 0; i1 < 4; i1++) {
        tmp[i1][i] = (*A)[i][i1];
     }
  }
  for (i = 0; i < 4; i++) {
     (*w)[i] = /* ScalProd */ ScalProd(&tmp[i], u);
  }
}
```

The product of two matrices

```
-- matrix product: A * B  (  (m,n) x (n,p)  )
function MatProd (A:real^m^n; B:real^n^p) returns (C:real^m^p)
   C = (map MatVectProd <<p>>)(A^p, B);
```

```
 /* MatProd */
void MatProd(
   /* MatProd::A */ array_real_4_3 *A,
   /* MatProd::B */ array_real_3_5 *B,
   /* MatProd::C */ array_real_4_5 *C)
{
  kcg_int i;

  for (i = 0; i < 5; i++) {
     /* MatVectProd */ MatVectProd(A, &(*B)[i], &(*C)[i]);
  }
}
```

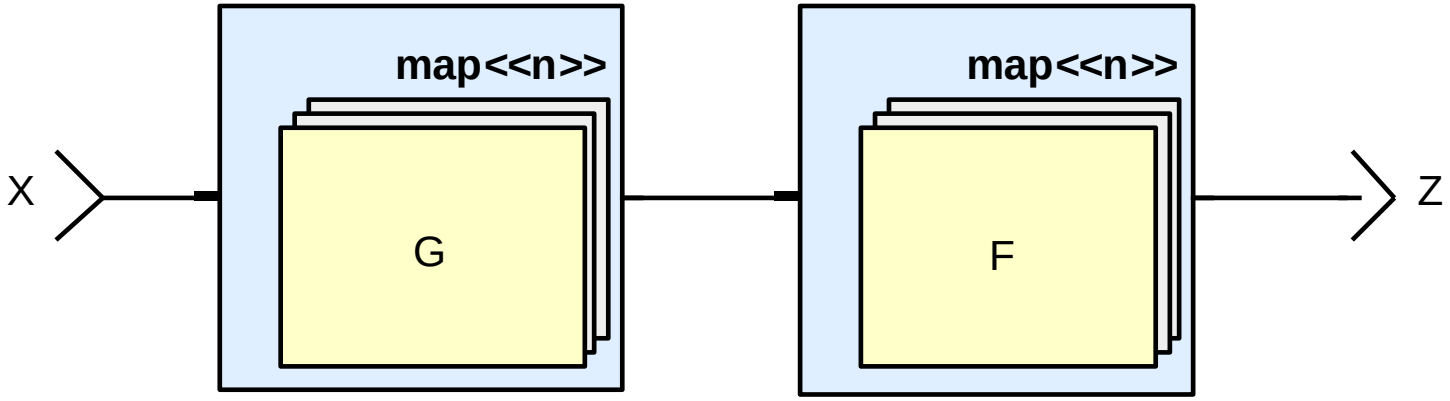
Optimization of arrays

- Simplifying remarkable identities :
  - (map F <<n>>) ((map G <<n>>) (X))

    = (map F <<n>>) ([G(X[0]) , … , G(X[n-1])])

    = [F(G(X[0])) , … , F(G(X[n-1]))]

    = (map (F o G) <<n>>) (X) ;



Optimization of arrays

- Do not build only iterated arrays

```
Y = (map F <<n>>) ( X ^ n) ;
```

```
for (i = 0; i < n; i++) {
    Y = F (X) ;   }
}
```

The product of two matrices: expanded

```
 /* MatProd */
void MatProd(inC_MatProd *inC, outC_MatProd *outC)
{
  kcg_int i;
  kcg_int i_ScalProd_MatVectProd;
  kcg_int i_MatVectProd;

  for (i = 0; i < 5; i++) {
    for (
      i_MatVectProd = 0;
      i_MatVectProd < 4;
      i_MatVectProd++)  {
      outC->C[i][i_MatVectProd] = 0.0;
     for (
       i_ScalProd_MatVectProd = 0;
       i_ScalProd_MatVectProd < 3;
       i_ScalProd_MatVectProd++)  {
       outC->C[i][i_MatVectProd] = outC->C[i][i_MatVectProd] +
            inC->A[i_ScalProd_MatVectProd][i_MatProd_MatVectProd] *
            inC->B[i][i_ScalProd_MatVectProd];
      }
    }
  }
}
```

The product of two matrices: expanded

```
 /* MatProd */
void MatProd(inC_MatProd *inC, outC_MatProd *outC)
{
  kcg_int i;
  kcg_int j;   /* i_ScalProd_MatVectProd; */
  kcg_int k;  /* i_MatVectProd; */

  for (i = 0; i < 5; i++) {
    for (k = 0; k < 4; k++) {
       outC->C[i][k] = 0.0;
       for (j = 0; j < 3;j++) {
         outC->C[i][k] = outC->C[i][k] + inC->A[j][k] * inC->B[i][j];
       }
     }
   }
}
```

There are still improvements to be made

- Copying arrays
  - voir A Modular Memory Optimization for Synchronous Data-Flow Languages. Application to Arrays in a Lustre Compiler.
- Merging loops in the imperative part

More Control

- Control: choose not to do a calculation
  - Performance(to put WCET into perspective)
  - Calculation is not always possible

- In a data flow context
  - Laws determined by operating ranges
  - Program purring but with discontinuities

- In Scade: only one construction **when** : Spartan !

- In Scade 6 :
  - Synchart style (Statechart – Stateflow®)
  - Complete mixture (Simulink-Stateflow or Esterel-Synchart)

Get rid of **current**

- **Current** : incompatible with initialization analysis

t	t_0	t_1	t_2	t_3	t_4	t_5	t_6	…
X = NAT()	0	1	2	3	4	5	6	…
Y = X when X mod 3 = 0	0			3			6	…
current Y	0	0	0	3	3	3	6	…

Get rid of **current**

- **Current** : incompatible with initialization analysis

t	t_0	t_1	t_2	t_3	t_4	t_5	t_6	...
X = NAT()	0	1	2	3	4	5	6	...
Y = X when X mod 3 = 0	0			3			6	...
Z = X when X mod 3 = 2			2			5		
current Y	0	0	0	3	3	3	6	...
current Z	nil	nil	2	2	2	5	5	...

Get rid of **current**

- **Current** : incompatible with initialization analysis

t	t_0	t_1	t_2	t_3	t_4	t_5	t_6	…
X = NAT()	0	1	2	3	4	5	6	…
Y = X when X mod 3 = 0	0			3			6	…
Z = X when X mod 3 = 2			2			5		
W = X when IN								
current Y	0	0	0	3	3	3	6	…
current Z	nil	nil	2	2	2	5	5	…
current W	nil	nil	nil	nil	nil	nil	nil	…

Replace it with **merge**

- **merge** :
  - merger of 2 flows on complementary clocks.

X = NAT()	0	1	2	3	4	5	6	…
Y = X when IN		1				5		…
Z = X when not IN	0		2	3	4		6	
Merge (IN; Y; Z)	0	1	2	3	4	5	6	

- Equivalent to If IN then current Y else current Z
  - But statically verifiable (less bug)
  - Poses no problem for initialization analysis

- Clocks are extended to enumerated

Program modes

- Pbm : According to a condition **C(IN)**, we have two laws **F1** and **F2** to calculate **OUT**.

- Using **merge** :

```
OUT1 = F1 (IN when cond) ;
OUT2 = F2 (IN when not cond) ;
OUT = merge (IN; OUT1; OUT2) ;
cond = C(IN) ;
```

  - It is necessary to :
    - Declare the different components of each variable
    - Associate clocks with all variables
  - It becomes complex when :
    - there are a lot of equations
    - there are nested submodes

Program modes

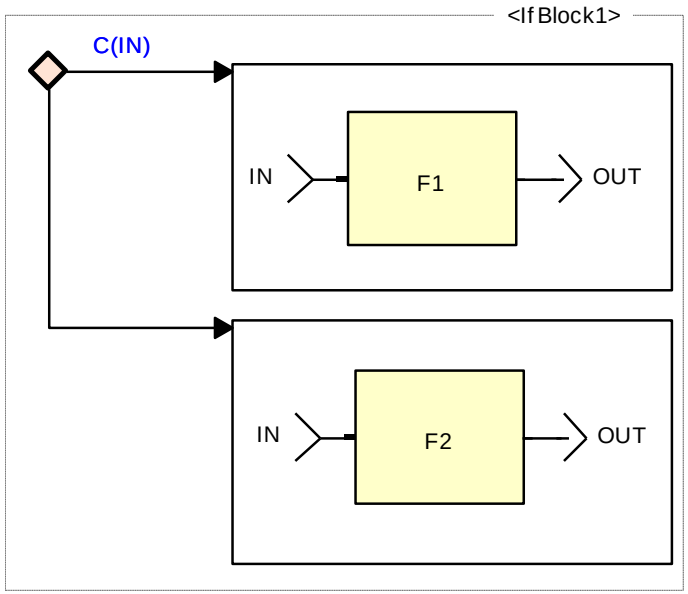
- Pbm : According to a condition **C(IN)**, we have two laws **F1** and **F2** to calculate **OUT**.

- Using **merge** :

```
OUT1 = F1 (IN when cond) ;
OUT2 = F2 (IN when not cond) ;
OUT = merge (IN; OUT1; OUT2) ;
cond = C(IN) ;
```

- We have a syntax :

```
activate if C(IN) then
   OUT = F1 (IN) ;
else
   OUT = F2 (IN) ;
returns OUT;
```



Compile modes

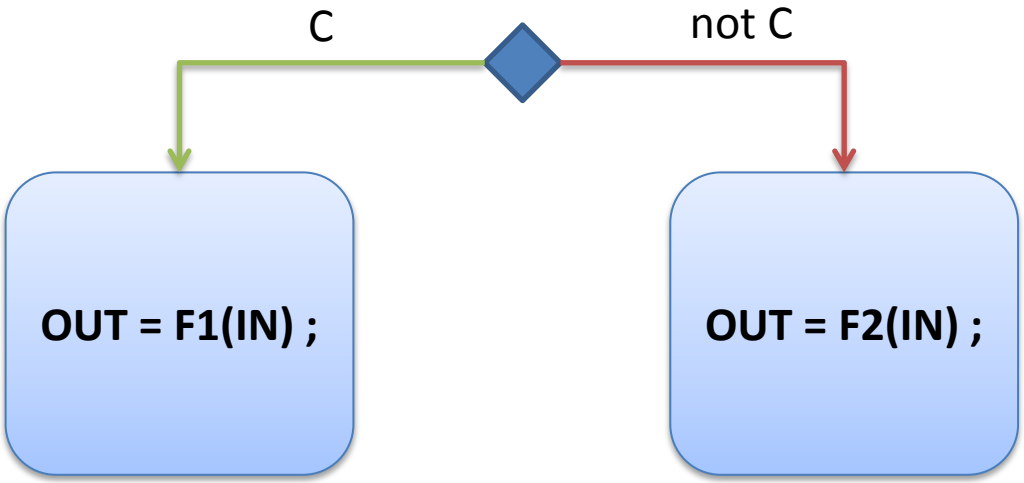
- The compilation of the modes follows exactly the diagram of the previous example :

```
activate if C(IN) then
   OUT = F1 (IN) ;
else
   OUT = F2 (IN) ;
returns OUT;
```

```
OUT1 = F1 (IN when cond) ;
OUT2 = F2 (IN when not cond) ;
OUT = merge (IN; OUT1; OUT2) ;
cond = C(IN) ;
```

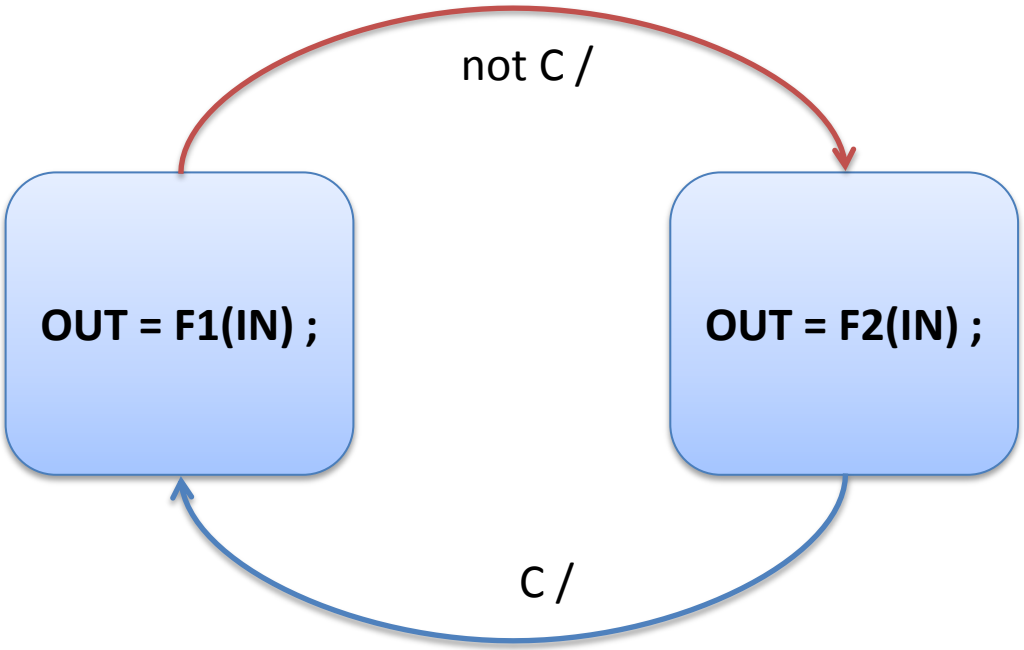
- The modes are expressed in the kernel language :
  - Conservative extension
  - Fully data flow

Modern automaton



- The condition is external to the modes.
- The active mode is chosen by the condition.

Modern automaton



- The controller is in one mode by default.
- We choose to stay or leave this mode.
- When to exit a mode :
    - Immediately (strong preemption)
    - The next instant (weak preemption)

Compile automata

- Necessary information (at each instant) :
  - Selected state
  - Strong transition fired
  - Active state
  - Weak transition fired
  - Next state selected

Compile automata

- Necessary information (at any instant) :
  - Selected state
  - Strong transition conditions
  - Strong transition fired
  - Action of fired transition
  - Active state
  - Body of active state
  - Weak transition fired
  - Action of fired transition
  - Next state selected

Not an order of evaluation
But a causal dependence

Compile automata

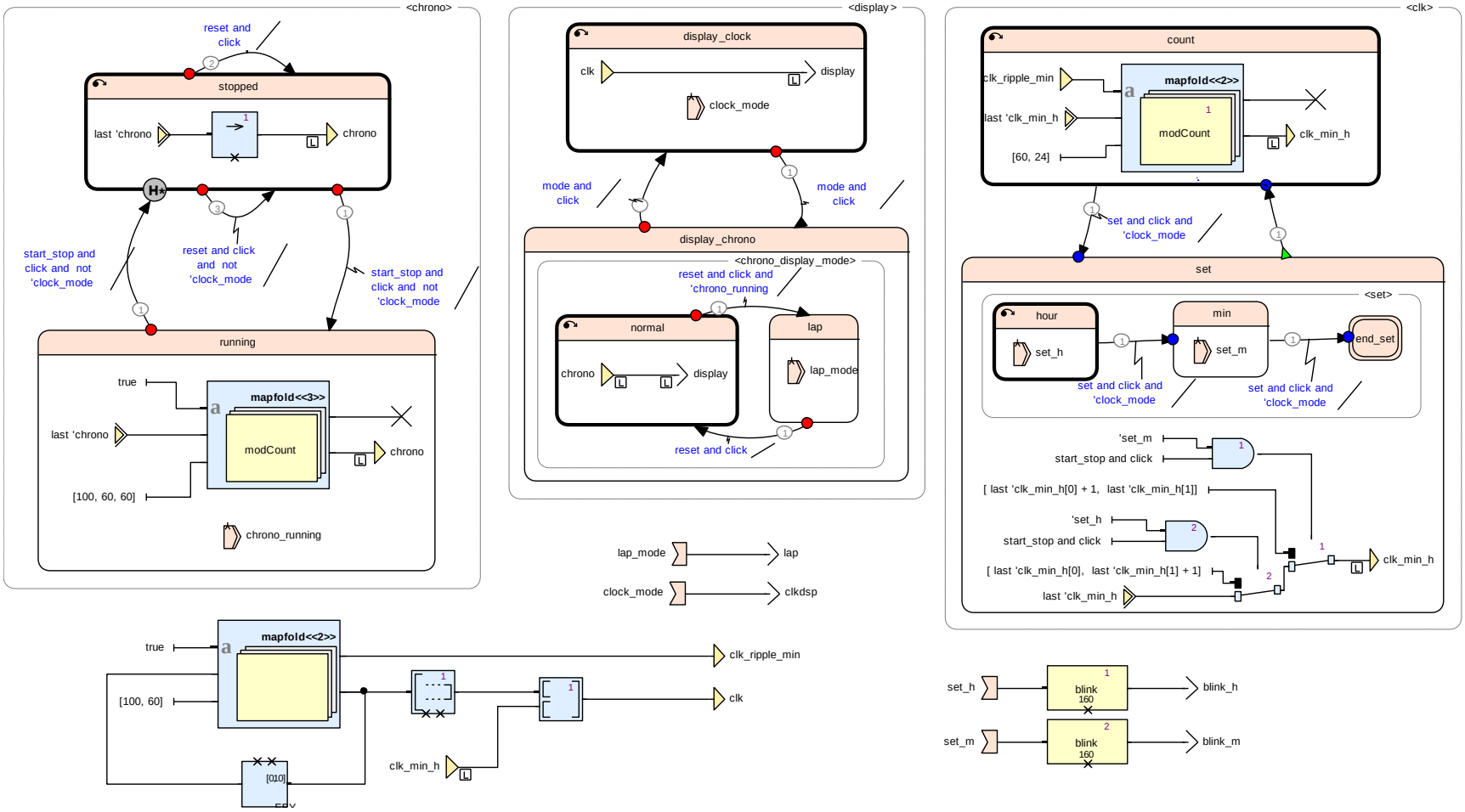
- Necessary information (at any instant) :
  - Selected state
  - Strong transition fired
  - Active state
  - Weak transition fired
  - Next state selected

The same value to the instant

$\Rightarrow$ Memory

$\Rightarrow$ state of the automaton

The digital watch: Scade 6 version



OCaml

- KCG is developed under the same conditions as co de intended to be embedded.

- Which programming language to use?

- OCAML :
  - With a simplified runtime
  - A coverage measurement tool

Voir Experience Report: Using Objective Caml to develop safety-critical embedded tool in certification framework.

References

- Type-based Initialization Analysis of a Synchronous Data-flow Language.
  Jean-Louis Colaço and Marc Pouzet. *International Journal on Software Tools for Technology Transfer (STTT)*, 2004

- Modular Causality in a Synchronous Stream Language.
  Pascal Cuoq and Marc Pouzet. (*ESOP'01)*

- Exploitation des structures régulières et des spécifications locales pour le développement correct de systèmes réactifs de grande taille.
  L. Morel, thèse de doctorat, 2005

- A Modular Memory Optimization for Synchronous Data-Flow Languages. Application to Arrays in a Lustre Compiler.
  Léonard Gérard, Adrien Guatto, Cédric Pasteur, and Marc Pouzet. *(LCTES'12)*

- Experience Report: Using Objective Caml to develop safety-critical embedded tool in certification framework.
  B. Pagano, O. Andrieu, T. Moniot, B. Canou, E. Chailloux, P. Wang, P. Manoury, J-L. Colaço. *(ICFP'09)*

Project Scade 6 Cast - KCG 6

- J-L. Colaço
- M. Pouzet
- F.X. Fornari

- N. Rostaing-Schmidt
- Y. Larvor
- O. Andrieu
- T. Moniot
- J. Verlaguet
- T. Rivière
- J. Forget

- G. Berry
- B. Dion

- F.X. Dormoy
- G. Collard
- L. Pouchan
- G. Maillard
- S.S. Lee

- Sorry to those I forget